

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 2 Duration : 1 Hour Total Marks:20

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I Nithya shree . k [192521145] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

NITHYA SHREE . K

CO1 Assessment: Constraint-Based Problem Solving – Answers

Answers to Question 4 and Question 7

4. Map Colouring of South Indian States (CSP with Heuristics)

(i) CSP formulation and constraint graph

Variables (X): TN, KL, KA, AP, TG, GA, MH (7 variables, one per state).

Domains (D): $D(\text{each variable}) = \{\text{Red, Green, Blue}\} = \{R, G, B\}$.

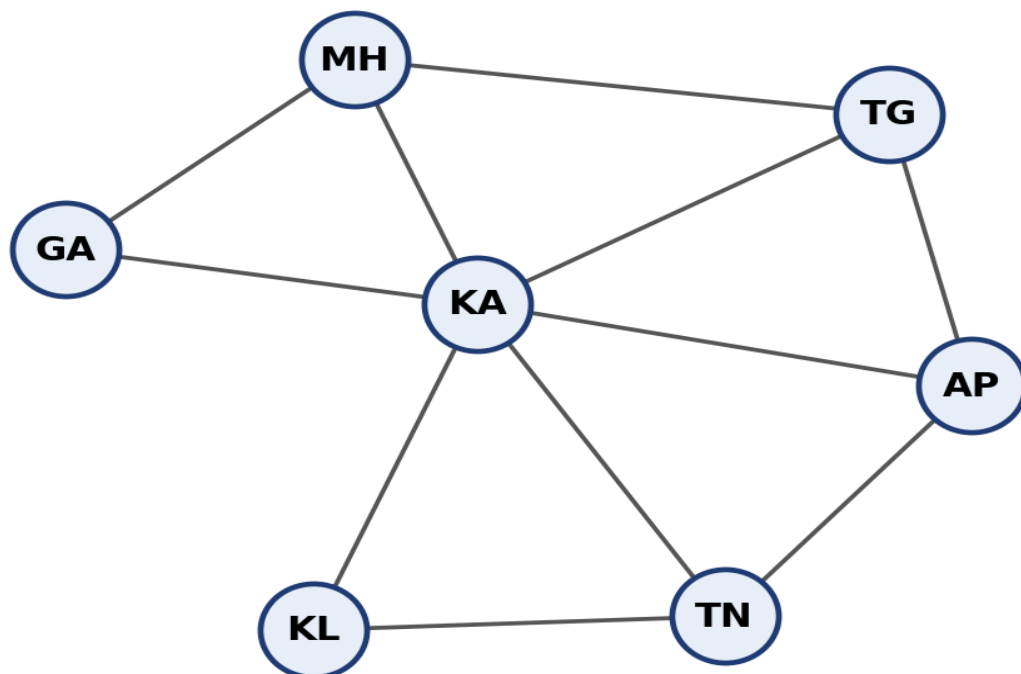
Constraints (C): Binary “not-equal” constraints, one per neighbouring pair (11 in total):

$TN \neq KL$, $TN \neq KA$, $TN \neq AP$, $KL \neq KA$, $KA \neq AP$, $KA \neq TG$, $KA \neq GA$, $KA \neq MH$, $AP \neq TG$, $GA \neq MH$, $MH \neq TG$

Constraint graph: each state is a node and each neighbouring pair is an edge.

Figure 1: Constraint graph of the map-colouring CSP

Constraint graph (7 nodes, 11 edges)



Degree of each node (number of neighbours):

State	KA	TN	AP	TG	MH	KL	GA
Degree	6	3	3	3	3	2	2
Neighbours	TN, KL, AP, TG, GA, MH	KL, KA, AP	TN, KA, TG	KA, AP, MH	KA, GA, TG	TN, KA	KA, MH

(ii) Variable-ordering heuristics

- **Degree heuristic (first variable):** KA has the highest degree (6), so it is the most constrained variable in the graph and is assigned first.
- **MRV (remaining variables):** at every later step choose the unassigned variable with the fewest legal values left in its domain. Ties are broken by the Degree heuristic (most constraints on other unassigned variables), then by list order.
- **Value ordering:** the colours are tried in the order R, G, B.

(iii) Backtracking with forward checking

Domains after each assignment (forward checking removes the assigned colour from the domains of unassigned neighbours):

Step	Variable chosen (reason)	Value	TN	KL	KA	AP	TG	GA	MH
0	Initial domains	–	RGB	RGB	RGB	RGB	RGB	RGB	RGB
1	KA (Degree = 6, highest)	R	G B	G B	R ✓	G B	G B	G B	G B
2	TN (MRV tie of 2; Degree tie-break, list order)	G	G ✓	B	R	B	G B	G B	G B
3	AP (MRV = 1 tie with KL; more unassigned neighbours)	B	G	B	R	B ✓	G	G B	G B
4	TG (MRV = 1; has unassigned neighbour MH)	G	G	B	R	B	G ✓	G B	B
5	MH (MRV = 1; has unassigned neighbour GA)	B	G	B	R	B	G	G	B ✓
6	KL (MRV = 1)	B	G	B ✓	R	B	G	G	B
7	GA (MRV = 1)	G	G	B	R	B	G	G ✓	B

Details of the checks:

- Step 1: KA = R removes R from all six neighbours (TN, KL, AP, TG, GA, MH). All of them now have {G, B}.
- Step 2: TN = G removes G from KL and AP, giving KL = {B} and AP = {B}. No domain is empty.
- Step 3: AP = B removes B from TG, giving TG = {G}. AP's other neighbours (TN, KA) are already assigned and consistent.
- Step 4: TG = G removes G from MH, giving MH = {B}.
- Step 5: MH = B removes B from GA, giving GA = {G}.
- Steps 6–7: KL and GA have a single legal value left, so they are assigned directly.

No domain became empty at any step, so **no backtracking** was needed.

(iv) Final colouring and effect of the heuristics

State	KA	TN	KL	AP	TG	MH	GA
Colour	Red	Green	Blue	Blue	Green	Blue	Green

Final colouring: Red = KA | Green = TN, TG, GA | Blue = KL, AP, MH

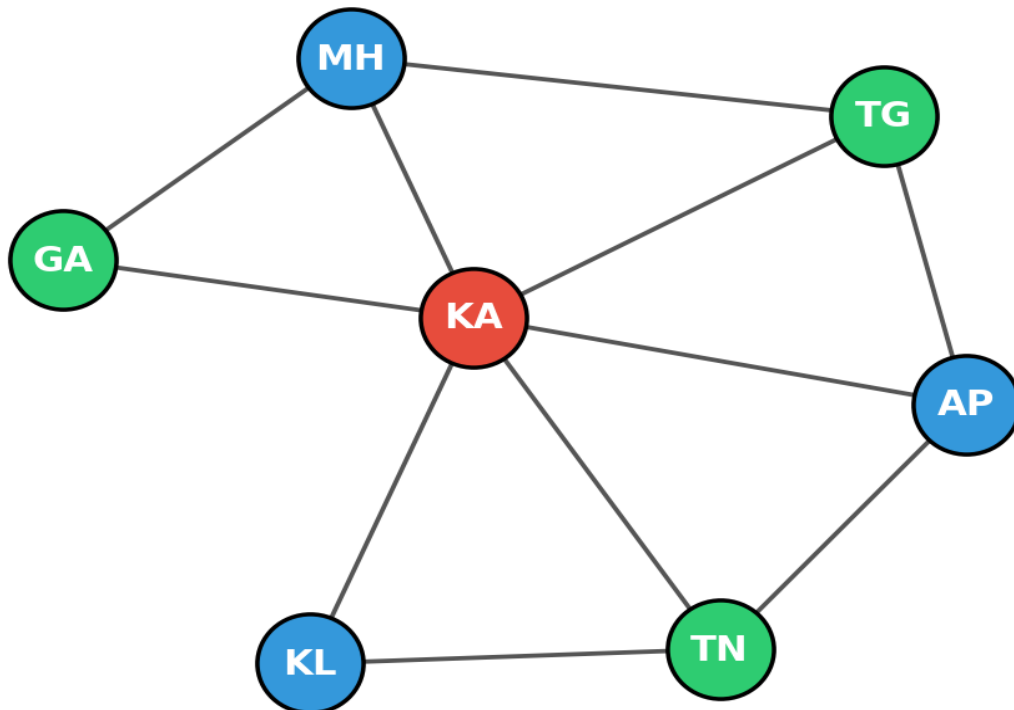


Figure 2: Final valid colouring

Verification of all 11 constraints: TN(G)–KL(B) ✓, TN(G)–KA(R) ✓, TN(G)–AP(B) ✓, KL(B)–KA(R) ✓, KA(R)–AP(B) ✓, KA(R)–TG(G) ✓, KA(R)–GA(G) ✓, KA(R)–MH(B)

✓, AP(B)–TG(G) ✓, GA(G)–MH(B) ✓, MH(B)–TG(G) ✓. No two neighbours share a colour.

How the heuristics reduced backtracking:

- The **Degree heuristic** picked KA first. KA touches six of the seven states, so fixing its colour early removed one colour from every other state at once and made the rest of the problem very small.
- **MRV** then always picked the variable with the smallest domain (a “fail-first” strategy). Single-value domains were assigned immediately, so any conflict would have shown up straight away instead of deep in the search tree.
- **Forward checking** passed each assignment on to the neighbours straight away and would have detected an empty domain at once. Here the chain $KA \rightarrow TN \rightarrow AP \rightarrow TG \rightarrow MH \rightarrow GA$ was forced, so the solution was found with **0 backtracks and only 7 assignments**. Without these heuristics a fixed alphabetical order could assign colours that fail later and force repeated backtracking.

7. Ambulance Routing During Floods (A* Search with a Road-Closure Constraint)

A* expands the node with the lowest $f(n) = g(n) + h(n)$, where g is the cost from S and h is the given heuristic. Roads are two-way, and the closed road **B–D** is removed from the graph

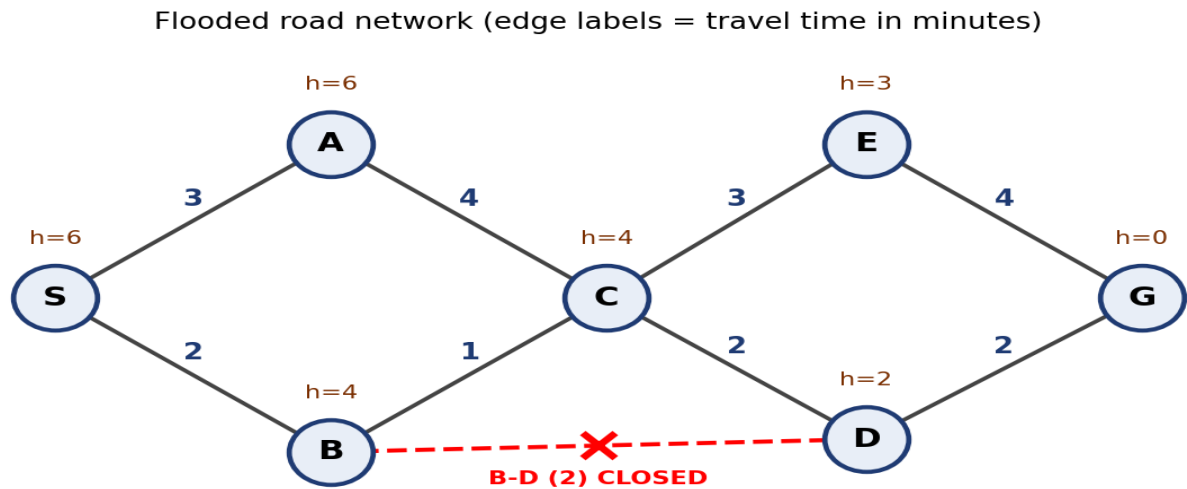


Figure 3: Flooded network with heuristic values

(i) A* search: open and closed lists

Entries are written as Node(g, h, f). The open list is kept sorted by f.

Step	Node expanded	Successors generated / updated	Open list (after step)	Closed list
0	—	Start: S(0, 6, 6)	S(0,6,6)	{ }
1	S (f= 6)	A(3, 6, 9) B(2, 4, 6)	B(2,4,6) A(3,6,9)	{S}
2	B (f= 6)	C(3, 4, 7) D not reachable: B–D is closed	C(3,4,7) A(3,6,9)	{S, B}
3	C (f= 7)	A via C: g = 7, f = 13, worse than 9, so discard D(5, 2, 7) E(6, 3, 9)	D(5,2,7) A(3,6,9) E(6,3,9)	{S, B, C}
4	D (f= 7)	G(7, 0, 7) C is already closed	G(7,0,7) A(3,6,9) E(6,3,9)	{S, B, C, D}

Step	Node expanded	Successors generated / updated	Open list (after step)	Closed list
5	G (f = 7)	Goal test passed, search stops	–	{S, B, C, D, G}

Nodes A and E stay in the open list with $f = 9$, which is larger than 7, so they are never expanded.

(ii) Route found and its cost

Path (by following parent pointers $G \rightarrow D \rightarrow C \rightarrow B \rightarrow S$):

$S \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

Cost = $2 + 1 + 2 + 2 = 7$ **minutes**. A* returns an optimal path because the heuristic is admissible (see part iv).

(iii) Route if road B–D were open

Step	Node expanded	Successors generated / updated	Open list (after step)	Closed list
0	–	S(0, 6, 6)	S(0,6,6)	{ }
1	S (f = 6)	A(3, 6, 9) B(2, 4, 6)	B(2,4,6) A(3,6,9)	{S}
2	B (f = 6)	C(3, 4, 7) D(4, 2, 6) [via B–D = 2]	D(4,2,6) C(3,4,7) A(3,6,9)	{S, B}
3	D (f = 6)	G(6, 0, 6) C via D: $g = 6$, $f = 10$, worse than 7, so discard	G(6,0,6) C(3,4,7) A(3,6,9)	{S, B, D}

Step	Node expanded	Successors generated / updated	Open list (after step)	Closed list
4	G (f = 6)	Goal test passed, search stops	–	{S, B, D, G}

With B–D open, A* returns $S \rightarrow B \rightarrow D \rightarrow G$ with cost $2 + 2 + 2 = 6$ minutes. The flood closure therefore adds **1 minute** to the best route (7 instead of 6) and changes the path from S–B–D–G to S–B–C–D–G.

(iv) Admissibility and consistency for the flooded network

Admissible means $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cheapest cost from n to G in the flooded network.

Node	S	A	B	C	D	E	G
$h(n)$ (given)	6	6	4	4	2	3	0
$h^*(n)$ (true cost)	7	8	5	4	2	4	0
Best path to G	S-B-C-D-G	A-C-D-G	B-C-D-G	C-D-G	D-G	E-G	–
$h \leq h^*$?	$\checkmark 6 \leq 7$	$\checkmark 6 \leq 8$	$\checkmark 4 \leq 5$	$\checkmark 4 \leq 4$	$\checkmark 2 \leq 2$	$\checkmark 3 \leq 4$	$\checkmark 0 \leq 0$

Since $h(n) \leq h^*(n)$ for every node, the heuristic is **admissible**.

Consistent means $h(n) \leq c(n, n') + h(n')$ for every edge. Because roads are two-way, both directions are checked:

Edge	c	$h(n) \leq c + h(n')$	Reverse: $h(n') \leq c + h(n)$	Result
S–A	3	$6 \leq 3 + 6 = 9$	$6 \leq 3 + 6 = 9$	✓
S–B	2	$6 \leq 2 + 4 = 6$ (tight)	$4 \leq 2 + 6 = 8$	✓
A–C	4	$6 \leq 4 + 4 = 8$	$4 \leq 4 + 6 = 10$	✓
B–C	1	$4 \leq 1 + 4 = 5$	$4 \leq 1 + 4 = 5$	✓
C–D	2	$4 \leq 2 + 2 = 4$ (tight)	$2 \leq 2 + 4 = 6$	✓
C–E	3	$4 \leq 3 + 3 = 6$	$3 \leq 3 + 4 = 7$	✓
D–G	2	$2 \leq 2 + 0 = 2$ (tight)	$0 \leq 2 + 2 = 4$	✓
E–G	4	$3 \leq 4 + 0 = 4$	$0 \leq 4 + 3 = 7$	✓

The inequality holds on all 8 roads in both directions, so the heuristic is **consistent** (and therefore also admissible). Consistency guarantees that f never decreases along a path and that every node on the closed list already has its optimal g value. This is why A^* never needs to reopen a closed node here, and why the route S–B–C–D–G (7 min) is optimal.

Note: closing a road can only increase true costs, so a heuristic that was admissible for the open network stays admissible after the closure.